

# Raspberry Pi 5 Real OP-TEE Deployment

Engineering Report, Failure Log, and User Manual

pi5-optee project

August 2026

## Abstract

This document is the final report for bringing **real OP-TEE** to a Raspberry Pi 5. It covers architecture and memory layout, a short chronology of every major failure and retry, the hardened LAN build/deploy/test scripts, validation results (`xtest`), and a user manual for rebuild and redeploy. Page budget: at most 10 pages.

## 1 Introduction and Goals

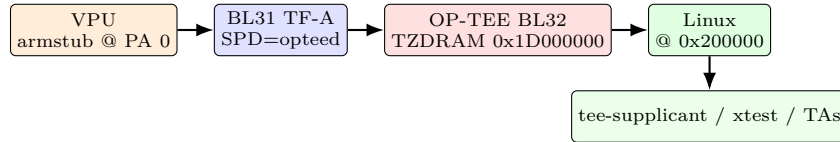
The goal was a Trusted Execution Environment on Pi 5 that is usable for real workloads and regression testing—not QEMU-only OP-TEE. That required: (1) an embedded BL31+OP-TEE armstub loaded by the VPU; (2) correct `kernel_address` so Linux does not destroy the stub; (3) a custom kernel with `CONFIG_TEE/CONFIG_OPTEE`; (4) userspace (`tee-suppllicant`, `libteec`, `TAs`, `xtest`); (5) LAN automation that does not destroy Wi-Fi configuration.

**Table 1:** Final software stack (validated on Pi 5).

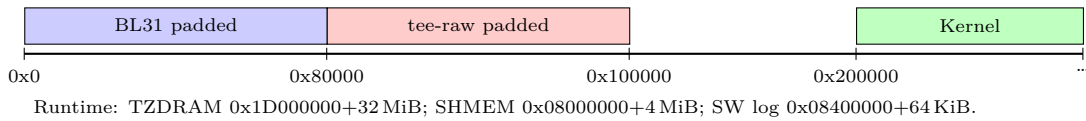
| Layer     | Component                                                                                   | Role                                       |
|-----------|---------------------------------------------------------------------------------------------|--------------------------------------------|
| Firmware  | <code>armstub8-2712-optee.bin</code> (1 MiB)                                                | BL31 + tee-raw embedded                    |
| Secure OS | OP-TEE OS ( <code>plat-rpi5</code> ), <code>CFG_DT=y</code>                                 | BL32 in TZDRAM; DT patched at boot         |
| Kernel    | <code>kernel_2712-optee.img</code>                                                          | TEE/OPTEE drivers → <code>/dev/tee*</code> |
| Userspace | <code>tee-suppllicant</code> , <code>libteec</code> , <code>xtest</code> , <code>TAs</code> | RPC + regression suite                     |
| Host      | Shell scripts under <code>~/pi5-optee</code>                                                | Build, LAN deploy, test, recover           |

## 2 Architecture

Figure 1 shows the boot handoff. Figure 2 shows why `kernel_address=0x200000` is mandatory: the embedded OP-TEE image occupies `[0x80000,0x100000)` inside the armstub until BL31 copies it to TZDRAM.



**Figure 1:** Pi 5 OP-TEE boot chain.



**Figure 2:** Armstub physical layout and kernel load address.

**Table 2:** Required `/boot/firmware/config.txt` keys.

| Key                               | Value                                | Notes                                   |
|-----------------------------------|--------------------------------------|-----------------------------------------|
| <code>armstub</code>              | <code>armstub8-2712-optee.bin</code> | Never plain <code>bl31.bin</code> alone |
| <code>kernel_address</code>       | <code>0x200000</code>                | Avoids overwrite of embed region        |
| <code>arm_64bit</code>            | <code>1</code>                       | 64-bit kernel                           |
| <code>dtoverlay=optee-rpi5</code> | <code>disabled</code>                | Redundant with <code>CFG_DT=y</code>    |

### 3 Chronology: Failures and Tries

Table 3 compresses the engineering history (24–29 Aug 2026). Early firmware experiments repeatedly left the board unreachable; the team initially attributed this to “Wi-Fi deleted.” Investigation showed boot hangs from incorrect stubs/addresses, while NetworkManager profiles usually remained. Ethernet became the preferred path for first OP-TEE reboots. A fresh Raspberry Pi Imager image (29 Aug) plus LAN deploy scripts produced a clean validation run.

**Table 3:** Failure log (short) and corrective try.

| #  | Try / symptom                        | Cause                                 | Fix                                                      |
|----|--------------------------------------|---------------------------------------|----------------------------------------------------------|
| 1  | Believed 32-bit OS mandatory         | Doc confusion                         | 64-bit Bookworm is correct                               |
| 2  | SSH/Wi-Fi “lost” after stub          | Boot hang, not wiped NM               | Ethernet; boot FAT recovery; scripts never rewrite Wi-Fi |
| 3  | Used plain BL31 as armstub           | No embedded BL32                      | 1 MiB embedded armstub only                              |
| 4  | Kernel crashed OP-TEE                | Default load @ 0x80000                | <code>kernel_address=0x200000</code>                     |
| 5  | Bad reserved-memory / probe          | Overlay + <code>CFG_DT=y</code> clash | Comment out <code>dtoverlay=optee-rpi5</code>            |
| 6  | Blank <code>armstub=</code> remotely | <code>sudo -S</code> stole stdin      | Write keys via <code>bash -c args</code>                 |
| 7  | ETXTBSY on supplicant                | File busy while running               | Stop/kill before copy                                    |
| 8  | Modules under nested KREL            | <code>scp -r</code> nesting           | <code>rsync</code> ; skip dangling build link            |
| 9  | Default host was Wi-Fi IP            | Mixed script defaults                 | Default to Ethernet LAN IP                               |
| 10 | No <code>/dev/tee*</code>            | Stock kernel lacks TEE                | Custom OP-TEE kernel + modules                           |
| 11 | Imager then SD vs LAN copy           | Ops choice                            | LAN deploy after Imager                                  |
| 12 | <code>xtest 1033</code> failed       | Missing <code>*.plugin</code>         | Install plugin; stage in build                           |

**Figure 3:** From failed tries to validated stack.

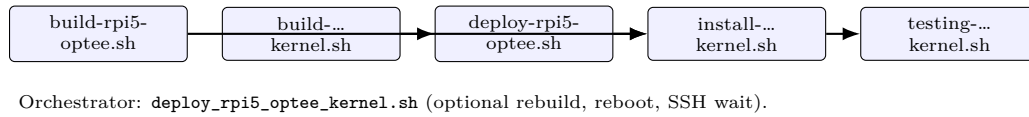
### 4 Tooling: Artifacts and Scripts

Artifacts land in `artifacts/rpi5-optee/` (armstub, tee\*.bin, kernel image, modules staging, rootfs overlay with `xtest/TAs/plugin`). Figure 4 shows the LAN pipeline.

### 5 Validation Results

Target after Imager + LAN deploy: `skmitra@192.168.0.107`.

- Kernel 6.18.46-v8-16k+; `/dev/tee0`, `/dev/teepriv0` present.
- tee-supPLICANT active; `eth0` connected.
- `config.txt`: correct `armstub` and `kernel_address`.
- `xtest -l 0`: 114 cases; one initial failure (1033) due to missing tee-supPLICANT plugin; after install, 1033 OK.
- Expected skips: pseudo-TA / fTPM / PAuth / MTE / some PTA cases.

**Figure 4:** Build → deploy → test pipeline.**Table 4:** Primary scripts.

| Script                                             | Purpose                                                     |
|----------------------------------------------------|-------------------------------------------------------------|
| <code>build-rpi5-optee.sh</code>                   | TF-A, OP-TEE OS, armstub, rootfs overlay (+ plugin)         |
| <code>build-rpi5-optee-kernel.sh</code>            | Image.gz + modules with OP-TEE fragment                     |
| <code>deploy-rpi5-optee.sh</code>                  | SCP armstub/userspace; safe <code>config.txt</code> updates |
| <code>install-rpi5-optee-kernel.sh</code>          | Install kernel + modules; password-aware reboot             |
| <code>deploy_rpi5_optee_kernel.sh</code>           | One-shot orchestration over LAN                             |
| <code>testing_rpi5_optee_kernel.sh</code>          | Smoke + <code>xtest</code>                                  |
| <code>monitoring_rpi5_optee_secure_world.sh</code> | NW command with SW log follow                               |
| <code>recover-boot-wifi.sh</code>                  | Offline boot FAT recovery helpers                           |

## 6 User Manual

### 6.1 Prerequisites

Build host (Debian/Ubuntu) with packages for TF-A/OP-TEE and kernel builds (`bc`, `bison`, `flex`, `libssl-dev`, ...). Pi 5 on **64-bit** Bookworm with SSH. Prefer Ethernet for first OP-TEE reboots. Set `export PI_SUDO_PASSWORD='...'` (or `SSHPASS`); do not hardcode secrets.

### 6.2 Build

```

cd ~/pi5-optee
OPTEE_MAX_LOG=1 ./build-rpi5-optee.sh
./build-rpi5-optee-kernel.sh # if kernel not built by the above

```

Sanity checks: armstub size = 1048576; OP-TEE region matches `tee-raw.bin` at offset 0x80000; overlay contains `xtest`, TAs, and `usr/lib/tee-supplciant/plugins/*.plugin`.

### 6.3 Deploy over LAN

```

export PI_SUDO_PASSWORD='...'
./deploy_rpi5_optee_kernel.sh --skip-build --skip-kernel-config-fetch \
  USER@PI_LAN_IP

```

Without `--skip-build`, the orchestrator rebuilds then deploys. Reboot is performed with password-aware `sudo`. Wait for SSH (Ethernet and known Wi-Fi IPs are both probed when defaults match the project hosts).

### 6.4 Test and monitor

```

./testing_rpi5_optee_kernel.sh USER@PI_LAN_IP
./monitoring_rpi5_optee_secure_world.sh USER@PI_LAN_IP -- xtest -l 0
# On device: sudo optee-sw-log -f

```

### 6.5 Recovery

If SSH dies after enabling the stub, treat it as a **boot hang**:

1. Connect Ethernet; try the LAN address.
2. Mount the boot FAT elsewhere; disable `armstub=` or rename the stub; restore from `/boot/firmware.file-backups/`.
3. Run `recover-boot-wifi.sh` on the SD boot partition.

### 6.6 Checklists and troubleshooting

## 7 Lessons Learned

- Pi 5 needs an **embedded** armstub and a non-default kernel load address; TF-A alone is not enough.

**Table 5:** `xtest -l 0` summary.

| Metric                    | Value                              |
|---------------------------|------------------------------------|
| Regression cases          | 114                                |
| Failures after plugin fix | 0                                  |
| Initial sole failure      | regression_1033 (plugin not found) |

**Table 6:** Pre-reboot checklist.

| Item                    | OK criterion                                                        |
|-------------------------|---------------------------------------------------------------------|
| Armstub                 | 1 MiB; embeds current <code>tee-raw</code>                          |
| <code>config.txt</code> | Correct <code>armstub</code> + <code>kernel_address=0x200000</code> |
| DT overlay              | <code>optee-rpi5</code> disabled                                    |
| Kernel/modules          | Matching OP-TEE Image and<br><code>/lib/modules/\$KREL</code>       |
| Plugin                  | Present under tee-supPLICANT plugins dir                            |
| Network                 | Ethernet path verified before reboot                                |

- Reachability loss after OP-TEE experiments was usually a **bad boot**, not deleted Wi-Fi—automation must never rewrite NM/netplan casually.
- Small automation defects (`sudo -S stdin`, `scp -r nesting`) can silently blank firmware keys or break module trees.
- Stock Raspberry Pi kernels omit TEE drivers; custom OP-TEE kernels are required for `/dev/tee*` and meaningful `xtest` coverage.

## 8 Appendix: Runtime Addresses

**Handoff note:** BL32 entry uses opteed conventions (`arg0=MODE_RW_64`, DTB PA in `arg3`). Do not enable the legacy device-tree overlay when `CFG_DT=y` is set.

## Document control

Repository: `pi5-optee`. Sources: `scripts`, `MEMORY-LAYOUT.txt`, and sessions 24–29 Aug 2026. Compile: `tectonic rpi5-optee-manual.tex` or `pdflatex` twice. Limit:  $\leq 10$  pages.

**Table 7:** Troubleshooting quick reference.

| Symptom                       | Likely action                                                         |
|-------------------------------|-----------------------------------------------------------------------|
| No <code>/dev/tee*</code>     | Install OP-TEE kernel; confirm <code>dmesg</code> OP-TEE probe        |
| Boot then silence             | Revert <code>armstub</code> ; check <code>kernel_address</code>       |
| <code>xtest 1033</code> fails | Install <code>*.plugin</code> ; restart <code>tee-supplciant</code>   |
| Modules missing deps          | Re-run install with <code>rsync</code> (not nested <code>scp</code> ) |
| ETXTBSY on copy               | <code>systemctl stop tee-supplciant</code> then retry                 |

**Table 8:** Runtime regions used by this port (see `MEMORY-LAYOUT.txt`).

| Region        | Base / size         | Purpose                                |
|---------------|---------------------|----------------------------------------|
| TZDRAM / BL32 | 0x1D000000 / 32 MiB | OP-TEE secure DRAM                     |
| Shared memory | 0x08000000 / 4 MiB  | NW↔SW buffers                          |
| SW log ring   | 0x08400000 / 64 KiB | <code>optee-sw-log</code> over SSH     |
| Kernel load   | 0x200000            | Avoids <code>armstub</code> OP-TEE pad |